



— COMPONENT DESIGN • PHASE 1

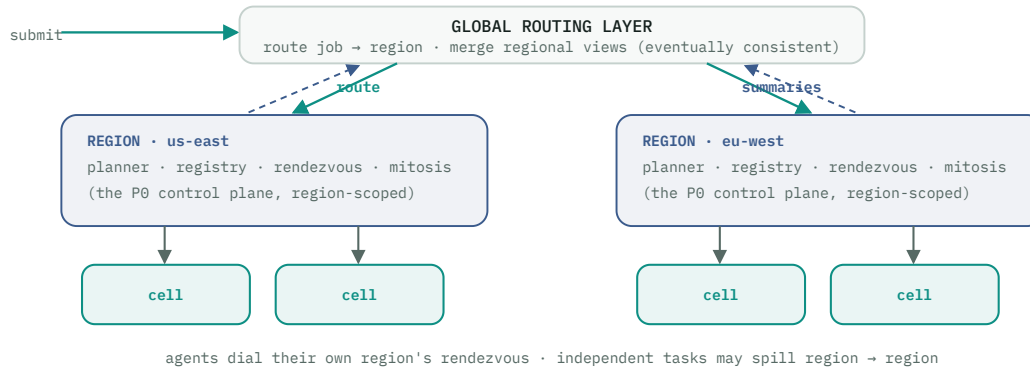
Multi-region, without moving the FCIS line.

P1 spreads the swarm across regions and shards the control plane per region, over a thin, **eventually-consistent** global layer. Every new piece is exactly what P0 promised: a new pure **core** (a decision) plus new **shell** I/O (cross-region transport). No P0 boundary is disturbed.

01 What changes from P0

P0 was one region. P1 makes the control plane **regional** — each region runs its own P0 control plane (planner, registry, rendezvous, mitosis) — and adds a thin **global routing layer** above them. Four deltas, no rewrites:

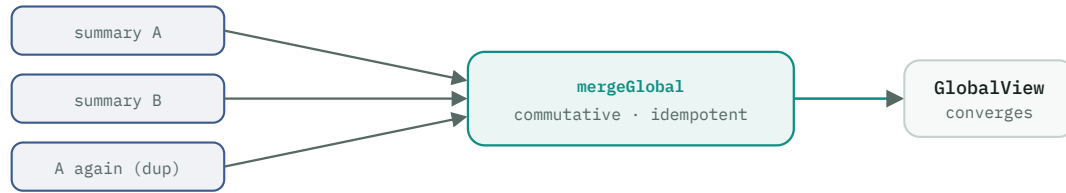
PIECE	P0	P1 DELTA
Control plane	single	one per region; publishes a regional summary upward
Routing	—	new global layer: route a job to a region, merge regional views
Placement	any local cell	may spill an independent task to a peer region when local is full
Agent failover	emits Failover	a decision fills it in: which region to dial next



The same P0 control plane, now one per region, under a thin router. Routing and the merged global view are new; everything inside a region is the P0 design unchanged. Regions push summaries up; the global layer never sits on the hot path.

02 The consistency model is a pure merge

Multi-region introduces the one new hazard P0 didn't have: regions disagree in transit. The design keeps this **off the hot path** (a region schedules from its own store — brief rule A1) and confines the reconciliation to a single pure function, `mergeGlobal`, that is **commutative, associative, and idempotent**. Those three laws are exactly what makes eventual consistency converge — and they're a gift to FCIS, because they're algebraic properties you can property-test without a network.



Order and duplicates don't matter, by law. Because the merge obeys the algebraic laws, summaries can arrive reordered or repeated and the global view still converges — the correctness of eventual consistency reduces to a property test on one pure function.

03 New & changed components

Global routing layer

GLOBAL · NEW

Responsibility — route each job to a region and hold the merged, eventually-consistent view of all regions.

◆ FUNCTIONAL CORE · PURE

```
func route(job JobSpec, regions []RegionView) Route
// Route = To{RegionID} | Spread{[]RegionID} | NoRegion
func mergeGlobal(v GlobalView, s RegionalSummary) GlobalView
// commutative · associative · idempotent
func diverged(v GlobalView, now Instant) []RegionID
// which regions look stale (for observability)
```

┘ IMPERATIVE SHELL · I/O

- receive `Submit` at the edge; forward to the routed region's planner
- gossip/replicate `RegionalSummary` across regions
- apply `mergeGlobal` ; serve the aggregate view

Boundary — `JobSpec` + `[]RegionView` in; a `Route` out. `mergeGlobal` 's laws are verified by property tests (feed shuffled, duplicated summaries → assert one fixed point), no cluster needed.

Regional control plane

REGION · DELTA FROM P0

Responsibility — the P0 planner/registry/rendezvous/mitosis, now scoped to one region and publishing a summary upward. The P0 cores are unchanged; they simply operate on this region's snapshot.

◆ FUNCTIONAL CORE · PURE

```
// all P0 cores unchanged: admit, place, decide, apply ...  
func summarize(reg Registry) RegionalSummary  
    // capacity · health · cell count for this region
```

▮ IMPERATIVE SHELL · I/O

- everything the P0 control plane did, tagged with a `RegionID`
- periodically publish `summarize()` to the global layer

Boundary — adds `RegionID` to identifiers; emits `RegionalSummary`. This is the proof of the FCIS promise: going regional added one pure function and one shell publish, and touched no existing core.

Cross-region placement

PLANNER • DELTA

Responsibility — for independent jobs, let a task spill to a peer region when the home region is full. Locality is preferred; only independent tasks may spill (tight jobs stay local in P2).

◆ FUNCTIONAL CORE • PURE

```
func placeAcross(t Task, local []CellView,  
                peers []RegionView) Placement  
    // prefer local; Spill{RegionID} when local full  
    // and a peer has capacity; else NoCapacity
```

▮ IMPERATIVE SHELL • I/O

- read local + peer snapshots (peers from the global view)
- on `Spill`, forward the task to the peer region's queue

Boundary — a task + local and peer snapshots in; a `Placement` (possibly `Spill`) out. Extends P0's `place`; the locality-first ranking is a pure, deterministic test.

Agent · cross-region failover

AGENT · DELTA

Responsibility — fill in the target of PO's `Failover` : when the home region won't answer, decide which region to dial next. The PO registration state machine is unchanged — this only supplies the destination.

◆ FUNCTIONAL CORE · PURE

```
func selectRegion(known []RegionID,
                  health map[RegionID]Health,
                  homeFirst bool, attempt int) RegionID
// prefer home, then nearest healthy peer
```

▀ IMPERATIVE SHELL · I/O

- keep the known-region list (cached from the global layer)
- on the PO `Failover` command, call `selectRegion` and dial it
- carry the SPIFFE identity across the region switch

Boundary — known regions + health + attempt in; a `RegionID` out. "Home down, two peers up, prefer nearest" is a one-line unit test, and the PO dial→enroll→join→heartbeat machine is reused verbatim.

04 The barrier spike (de-risking P2)

P1 also carries a **throwaway spike**: a standalone prototype of the future `barrier` coordinator, run against a *fake* network to prove the mechanics before P2 builds it for real. It ships nothing — but it's written FCIS-first, so what it proves transfers directly.

```
spike/barrier_core.go – the pure step, exercised with a fake shell
```

```
func step(s BarrierState, ev BarrierEvent, now Instant) (BarrierState, []BarrierEvent) {
    // BarrierEvent = Done{worker,partial} | Deadline | Tick
    // BarrierCmd    = AllReduce{partials} | Release{step} | Checkpoint | Evict{worker}
    // release step N+1 only when every worker has reported Done, or a Deadline event
```

Why spike it now. The barrier + checkpoint + mitosis-at-checkpoint interaction is the design's riskiest piece. Proving the pure step against a simulated set of workers (including an injected straggler and a mid-step loss) retires that risk in P1, so P2 is assembly, not discovery. The spike's core becomes the P2 core; only its fake shell is thrown away.

05 What P1 still defers

P1 is still **independent-driver, trusted, native** — it added reach, not capability. Tight coupling and the barrier/leader/message-passing drivers land in **P2** (the spike's core, productionized, plus checkpointing and gang scheduling); the WASM open tier and quorum/reputation verification land in **P3**; scale infrastructure in **P4**. Each remains, as here, a new pure core plus new shell I/O over the same boundary.